

RepoChat

Description:

RepoChat is an AI-powered developer tool that allows users to ask questions about any public GitHub repository. It takes in repository code, generates structured embeddings with metadata, and uses a Retrieval-Augmented Generation (RAG) pipeline to provide accurate explanations of the codebase with source references.

Technologies used:

- Frontend
 - Next.js
 - React
 - TypeScript
 - Tailwind
 - React Query / Fetch
 - Lucide React icons
- Backend
 - Node.js
 - Express
 - TypeScript
 - OpenAI SDK
 - Embedding model - text-embedding-3-small
 - Chat model - gpt-4o-mini
 - Tree-Sitter Parser (Concrete Syntax Tree (CST) parsing)
 - Git CLI (for repo cloning)
 - Winston (logging)
 - Env variable checker and validation on start
- Database
 - MongoDB (Atlas)
 - MongoDB Vector Search
- Security
 - HTTPS
 - Rate limiting (express-rate-limiter)
 - Input validation (ZOD)
 - OpenAI API key protection
 - Custom error handling

Project Setup:

- Root:
 - npm install npm@latest -g
 - mkdir repo-chat && cd repo-chat
 - git init
 - create .gitignore
 - npm init -y
 - create docker-compose.yml
 - configure eslint + prettier
 - create root scripts
 - create README
- Frontend:
 - cd frontend
 - npx create-next-app@latest
 - Install dependencies
 - tailwind
 - lucide-react
 - Create:
 - .env
 - .gitignore
 - Dockerfile
 - .dockerignore

- Configure:
 - Tailwind
 - Next.js environment variables
 - API base URL
- Backend:
 - cd backend
 - npm init -y
 - Install dependencies
 - express
 - cors
 - dotenv
 - mongoose
 - openai
 - tree-sitter
 - simple-git
 - nodemon
 - eslint
 - prettier
 - Create:
 - .env
 - .gitignore
 - Dockerfile
 - .dockerignore
 - Connect and test MongoDB
- DB:
 - MongoDB Atlas
 - Collections:
 - repositories
 - chunks
 - semantic_cache
 - Indexes:
 - Vector index on embeddings
- Cache
 - Redis
 - caches
 - Ratelimit
 - Previous questions

Supported Languages:

- TypeScript (.ts and .tsx)
- JavaScript (.js and .jsx)
- Python
- Java
- Go
- Ruby
- C++ (CPP)
- C# (C-Sharp)
- HTML
- CSS
- Markdown (Readme)

Project Management:

- GitHub for source control
- AWS for hosting
- GitHub Actions for CI/CD
- Draw.io for architecture diagrams
- Trello for development tracking
- Deployment:

- Elastic Container Registry - ECR
- Certificate Manager - ACM
- API Gateway
- CloudFront
- SSM Parameter Store
- Backend - Lambda
- Frontend - S3 (ECS Fargate in the future)
- Winston logs - CloudWatch
- Performance monitoring and stack trace - AWS X-Ray
- Redis - ElastiCache
- MongoDB - Atlas
- Containerised with Docker
- Logging using Winston

Testing Strategy:

- Backend
 - Jest
 - Supertest
 - Tests:
 - ingestion pipeline
 - vector search
 - RAG pipeline
 - API endpoints
- Frontend
 - React Testing Library
 - Cypress
 - Tests:
 - chat UI
 - repo loading
 - message flow

Repo Ingestion:

- Check repo size and DB space left before ingestion (make sure to add a multiplier since ingested data will be larger with the embedding and metadata)
- Repo size limit 25mb
- File type filtering exclude:
 - .git
 - node_modules
 - images
 - Assets
- Repos will be deleted when space is needed for newer repos
- Add a polling route to get live updates of the ingestion pipeline

Repo Query:

- Add a guard to catch malicious query use

Error Handling:

- What happens when the repo is too large?
 - The ingestion request gets rejected
- What happens when cloning fails?
 - A simple cloning failed message, along with try again later is sent to the frontend and then displayed
- What happens when OpenAI has errors?
 - Default all chat responses to a default apology/info message

Portfolio Enhancement Tips:

- Add example repositories for demo
- Include performance metrics
- Show architecture diagrams

- Show pipeline diagrams
- Add video demo
- For recruiters:
 - RAG architecture
 - AI engineering
 - Next.js full-stack systems

Documentation:

- ReadMe
- RAG pipeline explanation
- MongoDB Models
- API endpoints
- CI/CD explanation

Features:

Primary Features

- Repository ingestion
 - Input repo URL
 - Clone GitHub repo
 - Parse files
 - Chunk code using AST
- Vector database
 - Store code chunks
 - Store embeddings
 - Metadata filtering
- RAG query system
 - Query repository with natural language
 - Query interpreter
 - Vector search
 - Filtering
 - Reranking
 - Context compression
- Chat interface
 - Ask questions about the repo
 - Display answer
 - Show referenced files
- Semantic caching
 - Store previous answers
 - Reduce LLM cost
- Clean up
 - Repos are deleted only when more space is required

Secondary Features

- Source references
 - Show:
 - file path
 - function name
 - class name
- Repo file tree
 - Display repo structure.
- Question history
 - Show previous questions.
- Rate limiting
 - Prevent abuse
- Polling
 - Get repo ingestion updates

Communication Rules:

- Handlers can call
 - Controllers
- Controllers can call:
 - Multiple services
 - Format responses
 - Controllers should NOT:
 - Access the database directly
 - Call providers or repositories
- Services can call:
 - Repositories
 - Providers
 - Other services
 - Services should NOT:
 - Know about HTTP or Express
- Repositories can call
 - Database
- Providers can call
 - APIs
 - openAI

Clear Role Definitions:

- Handler
 - Role: API entry point.
 - Responsibilities:
 - Validate request
 - Parse inputs
 - Call controller
 - Return response
- Controller
 - Role: Orchestrate business logic.
 - Responsibilities:
 - Run RAG pipeline
 - Coordinate services
 - Format results
- Service
 - Role: Single-responsibility logic.
 - Examples:
 - cloneRepoService
 - chunkCodeService
 - vectorSearchService
 - rerankService
 - generateAnswerService
- Repositories
 - Role: a repository for each collection that will handle all MongoDB calls for that collection
 - Examples:
 - chunkRepository.ts
 - repoRepository.ts
 - cacheRepository.ts
- Repositories
 - Role: a repository for each api or external service
 - Examples:
 - completionProvider.ts
 - embeddingprovider.ts

File Layout:

project-root/

```
.gitignore
README.md
docker-compose.yml
package.json

backend/
|
|-Dockerfile
|-.dockerignore
|-package.json
|-.env
|-index.ts
|-.prettierrc
|-nodemon.json
|-repoCloning/
|-config/
|  |-config.ts
|  |-env.ts
|  |-initializeEnv.ts
|-error/
|  |-appErrors.ts
|-schemas/ (for ZOD)
|  |- ingest.schema.ts
|  |- query.schema.ts
|-types/
|
|-middleware/
|  |-errorHandler.ts
|  |-rateLimiter.ts
|  |-requestLogger.ts
|  |-responseHandler.ts
|  |-validate.ts
|
|-routes/
|  |-query.ts
|  |- ingest.ts
|  |- health.ts
|
|-controllers/
|  |-ragController.ts
|  |- ingestController.ts
|
|-services/
|
|-repositories/      (each repository corresponds to a MongoDB collection and holds all MongoDB queries for that collection)
|  |-chunkRepository.ts
|  |-repoRepository.ts
|  |-cacheRepository.ts
|  |-databaseRepository.ts
|
|-providers/      (Provider for AI interactions)
|  |-embeddingProvider.ts
|  |-completionProvider.ts
|
|-db/
|  |-models/
|  |  |-chunk.ts
```

- |- repo.ts
- |- mongoClient.ts
-
- |-lib/
 - |- logger.ts
 - |- ignoredDirs.ts
 - |- supportedLanguages.ts
-
- |- tests/

frontend/

-
- |-Dockerfile
- |-.dockerignore
- |-package.json
- |-.env
-
- |-app/
 - |-page.tsx
 - |- chat/
 - |-layout.tsx
-
- |-components/
-
- |-lib/
 - |- api.ts
-
- |-styles/
-
- |- tests/

database/

- |- Dockerfile
- |- init/
 - |- init-mongo.js
- |- scripts/
 - |- seed.js
 - |- backup.js
- |-logs/
- |-.env

docs/

- |-Plan doc
- |-System architecture
- |-RAG pipeline explanation
- |-MongoDB Models
- |-Setup guide
- |-Deployment guide
- |-API endpoints
- |-CI/CD explanation